

# Security Review

## AI Workspace Platform

### 1. Scope

This review covers the AI Workspace Platform backend (FastAPI + SQLAlchemy) and its interaction with the Streamlit frontend, focused on authentication, authorization, data isolation, API key handling, prompt injection, rate limiting, and sensitive data handling.

### 2. Risks, Mitigations, and Recommendations

| Risk                                                     | Severity | Current Mitigation                                                                                                                                     | Recommendation                                                                                                                    |
|----------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Weak/default SECRET_KEY in .env.example                  | High     | Documented requirement to generate a random key before production deploy; app does not hardcode a production key.                                      | Add a startup check that refuses to boot if SECRET_KEY equals the example placeholder in ENV=production.                          |
| Password brute-forcing on /api/auth/login                | Medium   | Rate limiting (10 requests/minute per IP) applied specifically to login and register endpoints via slowapi.                                            | Add account lockout after N failed attempts and CAPTCHA for repeated failures in production.                                      |
| JWT secret compromise                                    | High     | Tokens are short-lived (24h default, configurable) and signed with HS256; SECRET_KEY is environment-supplied, never committed.                         | Rotate SECRET_KEY periodically; consider RS256 with key rotation for multi-service deployments.                                   |
| Cross-workspace data access (broken authorization)       | High     | Every workspace-scoped route explicitly checks Workspace.owner_id == current_user.id before returning data; verified with an automated isolation test. | Add a matching test for every new route as the API surface grows; consider row-level security if migrating to PostgreSQL.         |
| Malicious file upload (executable disguised as document) | Medium   | Uploads are restricted to an explicit allow-list (pdf, docx, txt, md); extension is validated server-side, not trusted from Content-Type header alone. | Add magic-byte/content sniffing validation in addition to extension checks; scan uploads with an antivirus service in production. |
| Prompt injection via uploaded documents                  | Medium   | Grounded Q&A system prompt explicitly instructs the model to answer only from                                                                          | Add a content-based injection filter on ingested text (e.g. detect instructive phrasing like                                      |

|                                                                 |        |                                                                                                                                                   |                                                                                                                                                                      |
|-----------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                 |        | provided context and to say when the context lacks an answer, limiting (not eliminating) injected-instruction risk.                               | 'ignore previous instructions') before indexing.                                                                                                                     |
| API key exposure                                                | High   | API keys are read exclusively from environment variables via pydantic-settings; .env is git-ignored; no key is ever returned in any API response. | Use a secrets manager (e.g. Render/Railway secret env vars, AWS Secrets Manager) rather than plain .env files in production.                                         |
| Excessive data exposure in error messages                       | Low    | FastAPI's default validation errors are used as-is; HTTPException details are generic ('Workspace not found') rather than exposing internals.     | Disable DEBUG in production so stack traces are never returned to clients.                                                                                           |
| Denial of service via unbounded uploads                         | Medium | MAX_UPLOAD_MB setting exists in configuration for enforcement.                                                                                    | Wire MAX_UPLOAD_MB into an actual request-size-limiting middleware before production use (currently configuration-only).                                             |
| Sensitive data in logs                                          | Low    | Logging is limited to operational events (provider fallback warnings, startup); passwords/tokens are never logged.                                | Add a logging filter to redact Authorization headers if request/response logging is added later.                                                                     |
| Rate limiting bypass via distributed IPs                        | Medium | Current rate limiter is per-IP and in-process, which is bypassable by an attacker using many source IPs.                                          | For production, add authenticated-user-based rate limiting in addition to per-IP limiting, and move to a shared Redis-backed limiter for multi-instance deployments. |
| Conversation/document data at rest is unencrypted (SQLite file) | Medium | Acceptable for a development/coursework SQLite deployment; not exposed externally.                                                                | Use PostgreSQL with encryption at rest, or full-disk encryption on the host, for production.                                                                         |

### **3. Authentication Review**

Passwords are hashed with bcrypt (via Passlib) before storage; plaintext passwords never reach the database. JWT tokens are issued on login and required as a Bearer token on every protected route. Token validation failures return 401 with a WWW-Authenticate header, following the OAuth2 Bearer spec.

### **4. Authorization and Conversation Isolation Review**

Authorization is enforced consistently via a shared `_get_owned_workspace()` helper used by every workspace-scoped router (assistants, conversations, documents, memory, prompts, skills, dashboard, export). This was validated with an automated test asserting that a second user cannot list, read, or act on a first user's workspace, and that unauthorized access returns 404 rather than 403 to avoid leaking the existence of a resource to a caller who shouldn't know about it.

### **5. Rate Limiting**

A global default limit of 100 requests/minute per IP applies across the API, with a stricter 10 requests/minute limit specifically on `/api/auth/login` and `/api/auth/register` to slow credential-stuffing and account-enumeration attempts. Rate limiting is automatically disabled under the pytest suite (via a TESTING environment flag) so automated tests are not themselves rate-limited.

### **6. Sensitive Data and Secrets Handling**

All provider API keys, the JWT signing secret, and the database URL are supplied via environment variables through a single pydantic-settings Settings object. The `.env` file is excluded from version control via `.gitignore`, and only a placeholder `.env.example` is committed.

### **7. Summary Recommendation**

The current implementation is appropriate for a development/coursework deployment. Before any production or public deployment, the recommendations in Section 2 marked High or Medium severity should be addressed, particularly: enforcing a non-default `SECRET_KEY`, wiring `MAX_UPLOAD_MB` into actual middleware, and moving to a distributed rate limiter if deployed across multiple instances.